

Self-Balancing Two-Wheel Robot: Balancing on Uphill and Downhill Slopes

A Technical Report on the Engineering Problems, Root-Cause Analysis
and Control Solutions

Platform	ROS 2 Humble + Gazebo Classic 11 (ODE physics)
Controller	Cascade PD (velocity outer loop + pitch inner loop) with gravity feed-forward
Robot	Two-wheel inverted pendulum, IMU + wheel encoders, effort (torque) control
Scenarios	Flat road → 8° uphill ramp → 8° downhill decline
Date	15 June 2026

1. Project Overview

The project is a **self-balancing two-wheel robot** — an inverted pendulum on two wheels that must continuously drive its wheels to keep its body upright, exactly like a Segway. It is simulated in **ROS 2 Humble** with the **Gazebo Classic** physics engine. The robot reads its tilt from an on-board **IMU (gyroscope + orientation)** and its wheel speeds from the joint encoders, and commands a **torque** to each wheel.

The brain is a **cascade PD controller** with two nested loops:

- **Outer (velocity) loop:** compares the desired speed with the measured speed and produces a *target lean angle* (target pitch).
- **Inner (pitch / balance) loop:** drives wheel torque to make the actual tilt track that target lean — this is what keeps the robot from falling.

The goal of this work was to make the robot not just balance on flat ground, but also **drive up an 8° ramp** and **down an 8° decline** while staying balanced. The rest of this report walks through every significant problem we hit, what actually caused it, and how we fixed it.

2. Stage 1 — Core Self-Balancing Problems (Flat Ground)

Before any slope work, the basic balancer had to be made reliable. These were the key issues and fixes.

2.1 The robot appeared as a plain orange box

Problem: the robot's 3-D meshes did not load in Gazebo, showing a featureless box. Gazebo Classic could not reliably resolve the `package://mesh` URIs.

Fix: switched the mesh references to absolute `file://` paths, which Gazebo always resolves.

2.2 The robot kept toppling and spinning — the critical bug

Problem: the controller converts the IMU orientation (a quaternion) into a pitch angle. The original code used `math.asin()`, which is only valid for $\pm 90^\circ$. The instant the robot fell past 90° , the `asin` result *aliased back toward 0°* — so the controller believed the robot was upright, applied full torque the wrong way, and the robot spun out. This single line caused most of the early failures.

Fix: use `atan2()`, which is correct over the full $\pm 180^\circ$ range, so a falling robot is correctly seen as tilted.

```
# WRONG — aliases past 90 deg, robot thinks it is upright while falling
pitch = math.asin(sinp)

# FIXED — atan2 covers the full +/-180 deg range
def quat_to_pitch(qw, qx, qy, qz):
    sinp = 2.0 * (qw*qy - qz*qx)
    cosp = 1.0 - 2.0 * (qx*qx + qy*qy)
    return math.atan2(sinp, cosp)
```

Listing 2.1 — The pitch-angle fix. This was the root cause of the robot 'spinning out'.

2.3 The robot launched itself when reset ('snap upright')

Problem: Gazebo's effort controller *holds the last torque command*. When the balance node was killed, Python's cleanup (the `finally` block) was skipped by the SIGTERM signal, so a large stale torque stayed latched. On the next un-pause the robot was flung across the world.

Fix: explicitly publish a zero torque `[0.0, 0.0]` *before* killing the controller, so nothing is left latched.

2.4 Simulation-clock and process-storm issues

- **Timer stall:** resetting the simulation rewinds Gazebo's clock to 0, which froze the 200 Hz control timer. **Fix:** run the controller on the wall clock (`use_sim_time=False`) so resets cannot stall it.
- **Process storm:** launching with `respawn=True`, `respawn_delay=0.1` meant every crash spawned orphaned controller copies that kept respawning. **Fix:** removed `respawn` and always kill stale `gzserver/gzclient` processes (and clear `/dev/shm`) before each launch.

2.5 Residual balance oscillation

With the bugs gone the robot balanced, but with a $\pm 6^\circ$ limit-cycle wobble because the pitch gains ($k_p=120$, $k_d=15$) were a little stiff. This was tuned down later, and softer gains became important again on the slopes (Section 5).

3. Stage 2 — Uphill Slope (8° Ramp Climb)

The first slope scenario: a flat section, an **8° ramp going up**, then a raised plateau. The robot had to drive up the ramp and across the top while balancing.

3.1 The problem: tip-over at the ramp entry

Every attempt tipped the robot over right where the flat ground met the ramp. This was the single hardest problem of the uphill phase and resisted many attempts.

3.2 The wrong hypothesis (and why it was wrong)

Our first theory was a **physics 'double-contact' at the seam**: where the flat ground and the ramp overlapped, the ODE engine fired two contact points with different surface normals (flat vs 8°), producing a net forward kick that pitched the robot over. We tried ending the flat box just before the ramp — which made it worse, creating an 11 mm notch that the wheel dropped into and jammed against. The seam theory was a dead end.

3.3 The real root cause (found with high-rate logging)

Logging the controller at 20 Hz revealed the true failure chain:

- **The wheel jammed on the sharp concave corner** where the flat met the ramp. A round wheel rolling into a sharp mesh edge *catches* on it — a known ODE trimesh-vs-cylinder artifact. The robot stalled with full torque applied but zero forward motion.
- **Then a slip-driven runaway:** stalled, the body sagged back, the velocity loop drove the lean demand to its limit, torque saturated, the **wheels broke traction and spun up to ± 500 rad/s**. The encoders then reported a false ± 27 m/s, which fed back into the controller and kept the torque pinned — the robot tore itself over.

Lesson: the tip-over was not a physics impulse at the seam at all — it was a *geometry jam* followed by *wheel slip feeding the controller false data*.

3.4 The fix (three parts, all required)

- **One continuous collision surface.** The flat+ramp+plateau were replaced by a single `<polyline>` extrusion — one smooth mesh with no seam — and the concave corner was **rounded with a parabolic fillet** so there is no sharp edge to catch. This removed the jam.
- **Clamp the speed estimate.** The wheel-derived speed was clamped (± 0.6 m/s) so the absurd slip readings could no longer run the controller away.
- **Gravity feed-forward + gentler outer loop.** A torque integrator learns the steady force gravity applies on the slope and cancels it directly, so the robot keeps a small natural lean instead of riding the lean limit.

```
# Gravity feed-forward: learns the holding torque the slope needs.
# Sign-free -- it integrates the speed error, so it works up AND down a slope.
tau_ff += ki_tau * v_err * dt          # v_err = commanded_speed - measured_speed
tau_ff = clamp(tau_ff, -max_tau_ff, +max_tau_ff)
tau_balance = kp_p * pitch_err + kd_p * pitch_rate + 0.5 * tau_ff
```

Listing 3.1 — The gravity feed-forward term. It became central (and tricky) on the downhill.

Result: the climb was **solved** — the robot drives smoothly up the 8° ramp and onto the plateau, upright.

3.5 What we could NOT fully achieve uphill — and why

The climb worked, but the **final transition at the top — cresting the ramp and coming to a clean stop on the plateau — never became perfectly reliable** (best result: one tip during the stop). This was the outcome we did not fully reach on the uphill course.

Cause: at the crest the robot must instantly switch from 'leaning into an 8° hill with extra feed-forward torque' to 'vertical with zero torque on flat'. Two effects fought this transient:

- the **hard speed-clamp behaved as a discontinuity** at the crest overspeed, driving a limit-cycle (the robot hunting back and forth);
- the **feed-forward integrator re-wound** at the slope discontinuity — the robot rolled back slightly, the integrator slammed the torque up, producing a forward lurch.

The honest conclusion was that this last transient needs **gain/slope-scheduled control or a proper state estimator (Kalman filter)**, not more manual tuning — a limitation we carried forward into the downhill work.

4. Why Move From Uphill to Downhill?

With the uphill climb solved, the natural next scenario was the **opposite case: an 8° decline**. This is not a trivial mirror image — it is a fundamentally different control problem:

Uphill (gravity opposes)	Downhill (gravity assists)
Gravity tries to stop / stall the robot. The danger is not climbing (stalling) and slipping.	Gravity tries to run the robot away faster and faster. The danger is over-speeding and not being able to brake.
The controller must add drive torque.	The controller must hold the robot back (a braking torque) while staying upright.

The downhill course is: a flat top, an **8° decline** (dropping 0.84 m), then a lower flat. The robot spawns on top, drives forward, rolls over the brow, descends, and levels out. Crucially, it self-balances against **gravity-vertical** (the IMU defines 'upright' by gravity), so balancing on the incline is automatic — the challenge is **speed control and braking**.

5. Stage 3 — Downhill: Four Problems and Their Solutions

Getting the robot down the decline took diagnosing four distinct root causes, each found by reading the controller's own high-rate logs (the wheel encoders are 'ground truth').

5.1 Problem 1 — Over-braking tip at the brow

Symptom: as the robot rolled over the brow (flat→decline) it briefly over-spiced, the feed-forward and velocity loop both slammed to maximum braking, the robot over-leaned backward and tipped (pitch reached −0.82 rad).

Fix: a **long, gentle brow** — instead of a sharp edge, the flat blends into the decline over a 2 m parabolic curve (radius ≈ 14 m), so the descent starts gradually. Combined with gentler gains and a slower feed-forward build-up (k_{i_tau} 12 $\rightarrow$ 5).

5.2 Problem 2 — Wheel-slip runaway

Symptom: a small disturbance made the controller command full torque; the wheels **broke traction and spun to ~ 400 rad/s** (a false 30 m/s reading), the speed error saturated, and the robot ran away into a tip — the same slip mechanism seen uphill.

Diagnosis: we computed the actual **traction limit** of a wheel and found our torque cap was sitting right at it, so any weight shift broke the wheels loose:

```
# Per-wheel traction budget (max torque the tyre can transmit before slipping):
#   tau_max = mu * (m*g / 2) * r
#           = 0.8 * (12 * 9.81 / 2) * 0.075  ~=  3.5 N.m per wheel
#
# We had max_torque = 8 (then 5, then 3.5) -- AT or ABOVE the limit -> slip.
max_torque = 2.5      # ~70% of traction -> the wheels can NEVER break loose
```

Listing 5.1 — Sizing the torque cap below the traction limit was the key to stopping slip.

Fix: cap the torque at **2.5 N·m** (about 70% of traction) and soften the inner loop (k_{p_pitch} 120 $\rightarrow$ 70) so a small tilt error no longer instantly saturates the wheels.

5.3 Problem 3 — The velocity loop cancels its own brake (the deep one)

Symptom: the robot accelerated down the slope even though a strong braking feed-forward was active. The ground-truth logs were revealing: the **wheels ran away with almost zero applied torque**.

Root cause: on a downslope the velocity loop asks for a *lean-back* to brake — but the robot physically cannot lean back on a downhill, so the inner loop sees a 'need to pitch forward' error and produces a **forward drive that exactly cancels the braking feed-forward**. Net torque ≈ 0 , and gravity wins.

```
tau_balance = kp_p * (pitch - target_pitch) + 0.5 * tau_ff
#           \_____ +3.2 N.m _____/      \_____ -2.0 N.m ____/
# velocity loop demands a lean it can't reach -> big forward pitch_err term
# (+3.2) CANCELS the gravity brake (-2.0) -> ~0 net torque -> runaway.
```

Listing 5.2 — Why the robot wouldn't brake: the velocity loop fights the feed-forward.

Fix: keep the speed estimate on a **tight clamp while driving** ($\max_v_est = 0.15$). That keeps the speed error small, so the velocity loop only asks for a tiny lean and stops cancelling the brake. The robot then **creeps down the slope under feed-forward braking**, dead-vertical.

5.4 Problem 4 — Wheels slip when braking at the bottom

Symptom: the descent was now clean, but at the foot the robot arrives at ~ 1.4 m/s, and when it tried to brake the wheels slipped (spun to $+170$ rad/s) even within the torque limit, tipping it.

Root cause: a Gazebo contact parameter, `maxVel`, was set to 0.5. This is the engine's contact-correction speed cap; at 0.5 it **throttles wheel grip above 0.5 m/s** — exactly the speed the robot reaches at the foot. The wheels simply could not grip to brake.

```
<!-- description/gazebo.xacro : wheel contact -->
<minDepth>0.001</minDepth>
<maxVel>100.0</maxVel>      <!-- was 0.5: it starved grip above 0.5 m/s -->
```

Listing 5.3 — Raising the contact `maxVel` restored wheel grip at speed and fixed the stop.

Alongside this we added **command-gated logic** so the braking behaves correctly through the whole run: the feed-forward *builds* braking while driving but *releases* it when told to stop, the wound-up velocity

integral is bled to zero at the halt, and the speed clamp *opens* at the stop (on flat ground, leaning back genuinely brakes) so the robot can shed its last speed.

5.5 Result

The robot now drives the flat, rolls over the brow, and descends the full 8° decline completely tip-free, holding its pitch within $\pm 3.2^\circ$ of vertical the entire way down, and arrives at the lower level standing upright, where it balances for ~30 seconds.

Remaining limitation (left by design): a slow ~0.05 m/s creep eventually tips it about 30 s after the dead stop. This is inherent to a *velocity-only* balancer — it has no notion of *position*, so it cannot stand perfectly still forever. Fixing it needs **position feedback (station-keeping)**, the same conclusion reached for the uphill plateau-stop — a control-architecture change, not parameter tuning.

6. Summary of All Issues

#	Problem	Root cause	Solution
1	Robot shown as orange box	package:// mesh URIs unresolved in Gazebo	Use file:// absolute paths
2	Robot topples & spins	asin() pitch aliases past 90° → thinks it's upright	Use atan2() (full ±180°)
3	Robot launches on reset	Effort controller latches stale torque (SIGTERM skips cleanup)	Publish zero torque before kill
4	200 Hz timer stalls	Sim-clock reset rewinds time	Run controller on wall clock
5	Orphan process storm	respawn=True re-spawns crashed nodes	Remove respawn; kill stale procs
6	±6° balance wobble	Pitch gains too stiff	Soften kp/kd
7	Uphill: tip at ramp foot	Wheel jams on sharp concave edge → slip runaway	Rounded single polyline + speed clamp + feed-forward
8	Uphill: stop on plateau not clean	Clamp discontinuity + feed-forward re-wind at crest	(Residual) needs slope-scheduled / Kalman control
9	Downhill: tip at the brow	Brief over-speed → over-braking lean	Long gentle brow + slower ki_tau + gentle gains
10	Downhill: slip runaway	Torque cap at/above traction limit	max_torque 2.5 (<traction); softer kp_pitch
11	Downhill: won't brake, runs away	Velocity loop's lean demand cancels the brake	Tight speed clamp while driving (max_v_est 0.15)
12	Downhill: slip at the stop	Gazebo wheel maxVel=0.5 starves grip >0.5 m/s	maxVel 0.5 → 100 + command-gated braking
13	Downhill: slow creep after stop	Velocity-only controller has no position hold	(Residual) add odom station-keeping

Rows in red mark the two outcomes that remain partially open (the uphill plateau-stop and the downhill post-stop hold); both need a control-architecture upgrade rather than tuning.

7. Conclusion and Future Work

Across the three stages the project moved from a robot that could not stay upright at all, to one that **balances on flat ground, climbs an 8° ramp, and descends an 8° decline while self-balancing the whole way**. The recurring engineering theme was that the dramatic failures (spinning, runaways, tips) were almost never where they first appeared — the real causes were a wrong trig function, a geometry jam, wheel slip feeding false data, and a physics-engine contact cap. Each was found by **logging the controller's internal signals at high rate and reading the ground truth**, not by guessing.

Future work. Both remaining items — a perfectly clean stop at the top of the ramp and an indefinite stand-still at the bottom of the decline — point to the same upgrade: give the controller a **position estimate (odometry-based station-keeping)** and/or **slope-scheduled gains with a Kalman state estimator**. These would let the robot hold a fixed spot instead of slowly drifting, completing the last few percent of the behaviour.

Report generated from the project's engineering logs and simulation runs. All quoted numbers (pitch ranges, torques, traction limit, slip speeds) are taken directly from the recorded controller telemetry.